Automata Theory

Finite Automata & Regular Languages

CS32022

By Thisara Watawana

05 August 2026

2. Finite Automata & Regular Languages

Exercise 01

Imagine a basic touch-sensitive flash-light. It starts OFF.

- Tap it once to LOW brightness.
- Tap it a second time to HIGH brightness.
- Tap it a third time it turns OFF.

Your tasks;

1. Identify the States & Action and draw the state diagram
2. Generate state transition table
3. Does the lamp need to remember how many total taps it has ever received in its lifetime to work correctly

2. Finite Automata & Regular Languages

Finite Automata Behaviour:

1. It processes an input string symbol by symbol, moving strictly left-to-right without backtracking.
2. It has no RAM, no stack, and no tape to write on. Its current state is the only memory it possesses.
3. For every input symbol it reads, it uses its current state to make an immediate, unambiguous jump to the next state.
4. After reading the final symbol, if the machine rests in a designated Accept State, it approves the input; otherwise, it Rejects.

2. Finite Automata & Regular Languages

Formal Definition:

A Deterministic Finite Automaton (DFA) is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q is a finite set of states.

Σ is a finite alphabet (a set of allowed input symbols).

$\delta : Q \times \Sigma \rightarrow Q$ is the transition function mapping a (state, symbol) pair to a new state.

$q_0 \in Q$ is the start state.

$F \subseteq Q$ is the set of accept states (also called final states).

2. Finite Automata & Regular Languages

Formal Definition:

Key Notation Details to Remember:

- **Domain of δ:** δ takes its input from the Cartesian product **Q × Σ**.
- **Transitions:** Every state in **Q** must have an outgoing transition for every symbol in **Σ**.
- **Element vs. Subset:** q_0 is a single element ($q_0 \in Q$), whereas F is a set ($F \subseteq Q$).
- **Set F:** Can be empty ($\varnothing$), contain one state, or contain all states.

2. Finite Automata & Regular Languages

Exercise 02:

We want to design a Deterministic Finite Automaton *M*odd over the alphabet Σ = {0, 1} that accepts a binary string if and only if it contains an odd number of 1s.

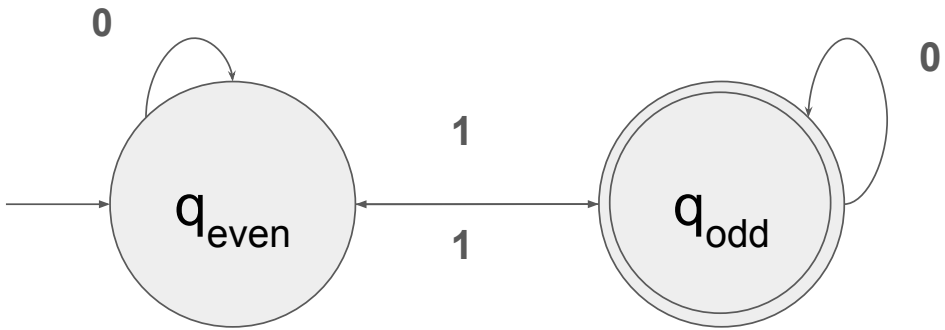
1. How many distinct memory conditions do you need to track whether the count of 1s seen so far is even or odd?
2. Draw the state diagram.
3. Write down Q, Σ, q_0, F, and construct the transition table δ.
4. Trace the input string w = 1011. Does your machine end in an accept state?

2. Finite Automata & Regular Languages

Exercise 02: State Diagram and Formal Definition

Formal 5-Tuple Specification:

- $\boldsymbol{Q} = \{q_{even}, q_{odd}\}$
- $\boldsymbol{\Sigma} = \{0, 1\}$
- $\boldsymbol{q_0} = q_{even}$ (0 is an even count of 1s)
- $\boldsymbol{F} = \{q_{odd}\}$



2. Finite Automata & Regular Languages

Exercise 02: Transition Table

Current State	Input 0	Input 1
q_{even} (Start)	q_{even}	q_{odd}
q_{odd} (Accept)	q_{odd}	q_{even}

String Trace Analysis

String: $w = 1011$

$r_0 = q_{even}$

$\downarrow 1$

$r_1 = q_{odd}$

$\downarrow 0$

$r_2 = q_{odd}$

$\downarrow 1$

$r_3 = q_{even}$

$\downarrow 1$

$\mathbf{r_4 = q_{odd}}$

Result: Since final state $r_4 = q_{odd} \in F$, the string 1011 is **ACCEPTED**.

2. Finite Automata & Regular Languages

Formal Definition of Computation

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a Deterministic Finite Automata (DFA) and let $w = w_1 w_2 \ldots w_n$ be a string over the alphabet Σ.

The machine M **accepts** w if a **sequence of states** $r_0, r_1, \ldots, r_n$ exists in Q satisfying the following three conditions:

1. **Start Condition:** $r_0 = q_0$
2. **Transition Condition:** $\delta(r_i, w_{i+1}) = r_{i+1}$ for $i = 0, \ldots, n-1$
3. **Accept Condition:** $r_n \in F$

Rejection: If $r_n \notin F$, the machine M **rejects** the string w.

2. Finite Automata & Regular Languages

Accept vs. Recognize

Action vs. Mapping

- A machine **accepts** or **rejects** an individual string w.
- A machine **recognizes** an entire language L(M).

The Language of a Machine

$L(M) = \{ w \in \Sigma^* \mid M \text{ accepts } w \}$

If M accepts no strings, it recognizes the empty language $L(M) = \varnothing$.

Regular Language

A language A is **regular** if some finite automaton recognizes A (meaning A = L(M) for some DFA M).

2. Finite Automata & Regular Languages

Why the Distinction Matters in Computer Science?

Acceptance is Local (Instance Level): *M* accepts *w* is an operational check on a single, finite input string *w*. It takes exactly |*w*| steps to execute.

Recognition is Global (Specification Level): *M* recognizes *L*(*M*) means the finite 5-tuple, its states *Q* and transition rules *δ,* captures the structural definition of the entire (often infinite) language. It provides a finite representation for an infinite set.

The Compiler Analogy: A compiler doesn't store a list of every valid code file ever written (which is infinite). Instead, the programming language's syntax is recognized by the compiler's finite parser. When a developer writes a specific code file *w*, the parser accepts or rejects that single string.

2. Finite Automata & Regular Languages

Exercise 03:

If the given alphabet is $\Sigma = \{a, b\}$, design a DFA M_1 that accepts any string whose length $|w|$ is a multiple of 3.

1. Determine the set of states Q, start state q_0, and accept set F. (Hint: Think about remainders when dividing string length by 3).
2. Sketch the state diagram and complete the transition table δ.
3. Trace the state sequence $r_0, r_1, ..., r_6$ for w = aaabbb. Does your machine end in F?

2. Finite Automata & Regular Languages

Exercise 04:

If the given Alphabet: $\Sigma = \{0, 1\}$, design a DFA M_2 that accepts strings that start with 10.

Instructions:

1. Identify all states Q, start state q_0, and accept set F. (Hint: What happens if the very first symbol is 0?)
2. Sketch the state diagram and complete the transition table δ.
3. Trace state sequences for both $w_1 = 1010$ and $w_2 = 0110$. Confirm that your machine accepts w_1 and rejects w_2.

2. Finite Automata & Regular Languages

Exercise 05:

If the given alphabet: $\Sigma = \{0, 1\}$, design a DFA M_3 that accepts strings that end with 01.

1. Identify states Q needed to track progress toward 01, specify start state q_0, and accept set F.
2. Sketch the state diagram and complete the transition table δ. (Hint: What happens if you get a 0 when expecting a 1?)
3. Trace the step-by-step state sequence for $w = 1001$. Does it land in an accept state?

2. Finite Automata & Regular Languages

Exercise 06:

If the given alphabet: $\Sigma = \{0, 1\}$, design a DFA M_4 that accepts strings containing the substring 11 anywhere.

1. Determine states Q, start state q_0, and accept set F. (Hint: Once 11 is found, does the machine ever need to leave the accept state?)
2. Sketch the state diagram and complete the transition table δ.
3. Trace the state sequence for $w = 0110$. Verify the exact step where your machine enters the accept state.

2. Finite Automata & Regular Languages

Exercise 03 - Answers:

The machine only needs to remember the remainder of the string length divided by 3.

State Design:

- q_0: Length mod 3 = 0 (Start state & Accept state)
- q_1: Length mod 3 = 1
- q_2: Length mod 3 = 2

Formal Solution:

- $Q = \{q_0, q_1, q_2\}$
- $q_0 = q_0$, $F = \{q_0\}$ (empty string ε has length 0, which is a multiple of 3)

Transitions:

Current State	Input a	Input b
q_0 (Start/Accept)	q_1	q_1
q_1	q_2	q_2
q_2	q_0	q_0

2. Finite Automata & Regular Languages

Exercise 04 - Answers:

The machine validates the first two characters. If a string violates the prefix rule early, it drops into a state it can never escape.

State Design:

- q_{start}: Reading symbol 1
- q_{saw_1}: Read 1, expecting 0
- q_{accept}: Successfully started with 10 (Accept state)
- q_{dead}: Violates prefix rule (Rejection trap)

Formal Solution:

- $Q = \{q_{start}, q_{saw_1}, q_{accept}, q_{dead}\}$
- $q_0 = q_{start}$, $F = \{q_{accept}\}$

Transitions:

Current State	Input 0	Input 1
q_{start}	q_{dead}	q_{saw_1}
q_{saw_1}	q_{accept}	q_{dead}
q_{accept} (Accept)	q_{accept}	q_{accept}
q_{dead}	q_{dead}	q_{dead}

2. Finite Automata & Regular Languages

Exercise 05 - Answers:

The machine cannot predict when the string ends, so it must always track whether the most recent symbols read match the target pattern 01.

State Design:

- q_{start}: Base state / Last symbol read was not 0
- q_0: Last symbol read was 0 (halfway to target)
- q_{01}: Last two symbols read were 01 (Accept state)

Formal Solution:

- $Q = \{q_{start}, q_0, q_{01}\}$
- $q_{start} = q_{start}$, $F = \{q_{01}\}$

Transitions:

Current State	Input 0	Input 1
q_{start}	q_0	q_{start}
q_0	q_0	q_{01}
q_{01} (Accept)	q_0	q_{start}

2. Finite Automata & Regular Languages

Exercise 06 - Answers:

Once the target pattern 11 is found anywhere in the string, the machine has succeeded and remains in the accept state regardless of future inputs.

State Design:

- q_{base}: Have not seen 1 yet
- q_{saw_1}: Saw a single 1
- q_{found}: Saw 11 (Accept state)

Formal Solution:

- $Q = \{q_{base}, q_{saw_1}, q_{found}\}$
- $q_0 = q_{base}$, $F = \{q_{found}\}$

Transitions:

Current State	Input 0	Input 1
$\boldsymbol{q_{base}}$	q_{base}	q_{saw_1}
$\boldsymbol{q_{saw_1}}$	q_{base}	q_{found}
$\boldsymbol{q_{found}}$ (Accept)	q_{found}	q_{found}

Questions?

thisaraw@boffin.lk